```
Start coding or generate with AI.
```

Matplotlib

Matplotlib is a low level graph plotting library in python that serves as a visualization utility.

Matplotlib was created by John D. Hunter.

Matplotlib is open source and we can use it freely.

Matplotlib is mostly written in python, a few segments are written in C, Objective-C and Javascript for Platform compatibility

With Matplotlib, we can perform a wide range of visualization tasks, including:

Creating basic plots such as line, bar and scatter plots.

Customizing plots with labels, titles, legends and color schemes.

Adjusting figure size, layout and aspect ratios.

Saving plots in various formats like PNG, PDF and SVG.

Combining multiple plots into subplots for better data representation.

Creating interactive plots using the widget module.

Installation of Matplotlib

pip install matplotlib

Import Matplotlib

Once Matplotlib is installed, import it in your applications by adding the import module statement:

import matplotlib

Matplotlib Pyplot

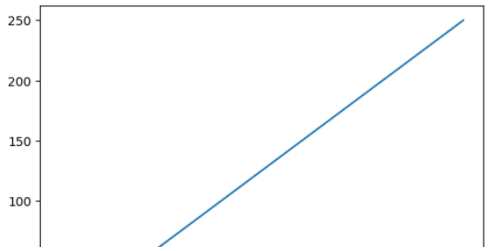
Pyplot

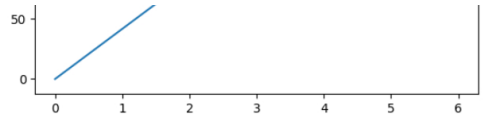
Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

```
import matplotlib.pyplot as plt
```

Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([0, 6])
ypoints = np.array([0, 250])

plt.plot(xpoints, ypoints)
plt.show()
```





Double-click (or enter) to edit

Matplotlib Plotting

Plotting x and y points

The plot() function is used to draw points (markers) in a diagram.

By default, the plot() function draws a line from point to point.

The function takes parameters for specifying points in the diagram.

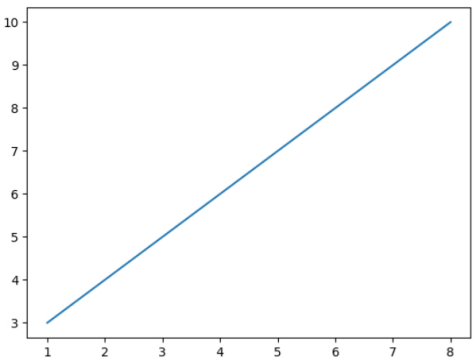
Parameter 1 is an array containing the points on the x-axis.

Parameter 2 is an array containing the points on the y-axis.

If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

Draw a line in a diagram from position (1, 3) to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```



The x-axis is the horizontal axis.

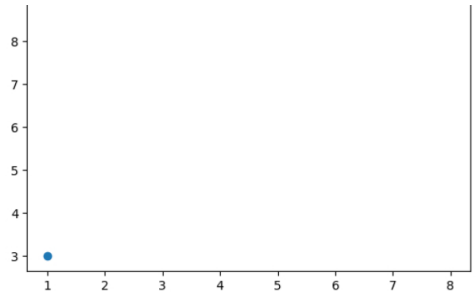
The y-axis is the vertical axis.

Plotting Without Line

To plot only the markers, you can use shortcut string notation parameter 'o', which means 'rings'.

Draw two points in the diagram, one at position (1, 3) and one in position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np

xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints, 'o')
plt.show()
```



Double-click (or enter) to edit

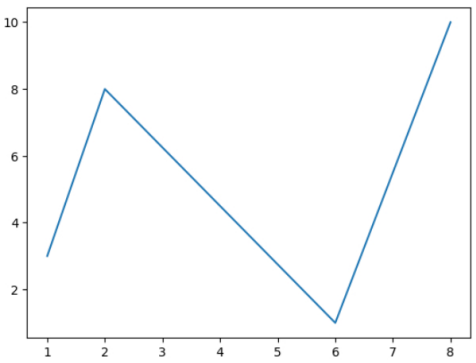
Multiple Points

We can plot as many points as you like, just make sure you have the same number of points in both axis.

Double-click (or enter) to edit

Draw a line in a diagram from position (1, 3) to (2, 8) then to (6, 1) and finally to position (8, 10):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])
plt.plot(xpoints, ypoints)
plt.show()
```



Default X-Points

If we do not specify the points on the x-axis, they will get the default values 0, 1, 2, 3 etc., depending on the length of the y-points.

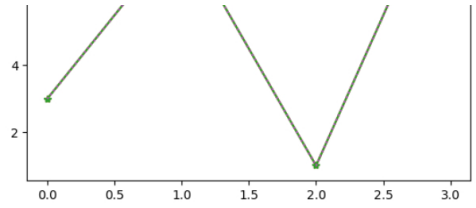
So, if we take the same example as above, and leave out the x-points, the diagram will look like this:

Plotting without x-points:

Use a dotted line:

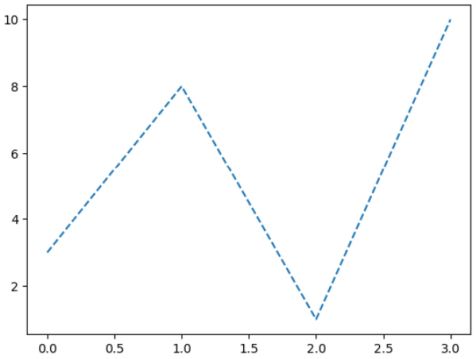
Double-click (or enter) to edit

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
ypoints = np.array([3, 8, 1, 10])
plt.plot(ypoints, linestyle = 'dotted')
plt.show()
```



Use a dashed line:

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
ypoints = np.array([3, 8, 1, 10])
plt.plot(ypoints, linestyle = 'dashed')
plt.show()
```



Shorter Syntax

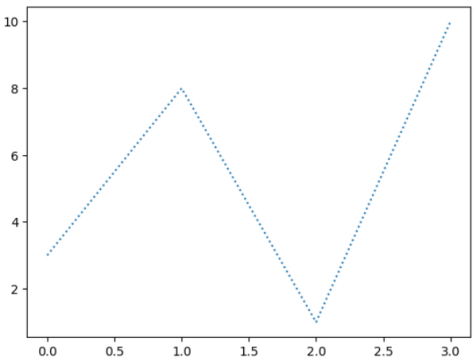
The line style can be written in a shorter syntax:

linestyle can be written as ls.

dotted can be written as :.

dashed can be written as --.

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
ypoints = np.array([3, 8, 1, 10])
plt.plot(ypoints, ls = ':')
plt.show()
```




```
Start coding or generate with AI.
```

Matplotlib Labels and Title

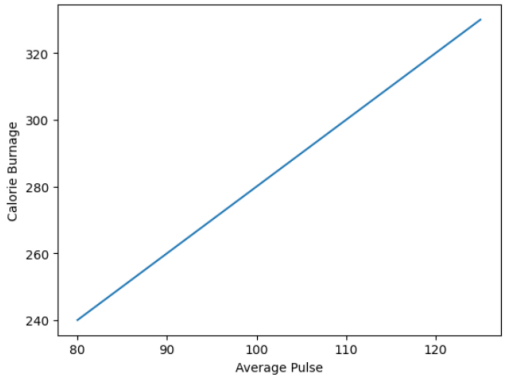
Create Labels for a Plot

With Pyplot, we can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

Add labels to the x- and y-axis:

Add labels to the x- and y-axis.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

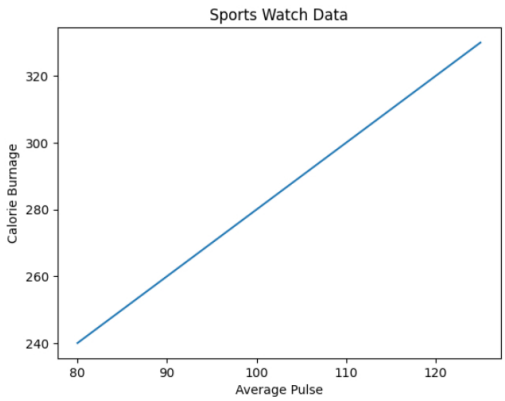


Create a Title for a Plot

With Pyplot, you can use the title() function to set a title for the plot.

Add a plot title and labels for the x- and y-axis:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.plot(x, y)

plt.title("Sports Watch Data")
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.show()
```

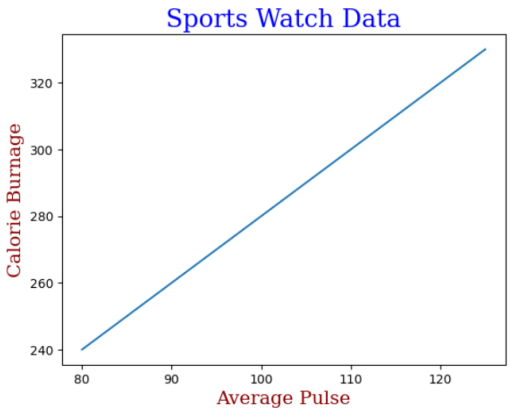


Set Font Properties for Title and Labels

We can use the fontdict parameter in xlabel(), ylabel(), and title() to set font properties for the title and labels.

Set font properties for the title and labels:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}

plt.title("Sports Watch Data", fontdict = font1)
plt.xlabel("Average Pulse", fontdict = font2)
plt.ylabel("Calorie Burnage", fontdict = font2)

plt.plot(x, y)
plt.show()
```



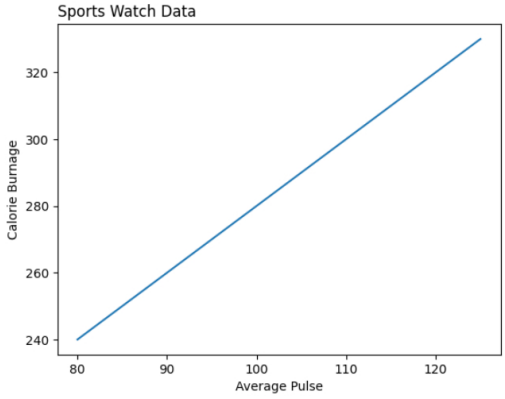
Position the Title

We can use the loc parameter in title() to position the title.

Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

Position the title to the left:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])

plt.title("Sports Watch Data", loc = 'left')
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")

plt.plot(x, y)
plt.show()
```



Matplotlib Scatter

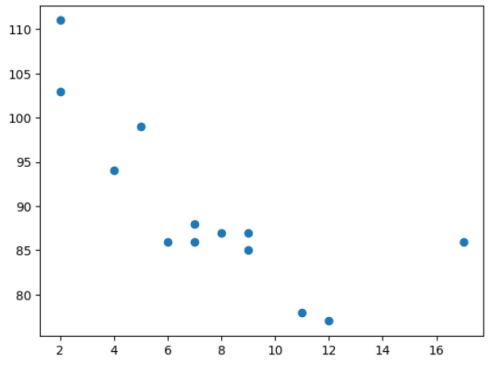
Creating Scatter Plots

With Pyplot, you can use the scatter() function to draw a scatter plot.

The scatter() function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

A simple scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```



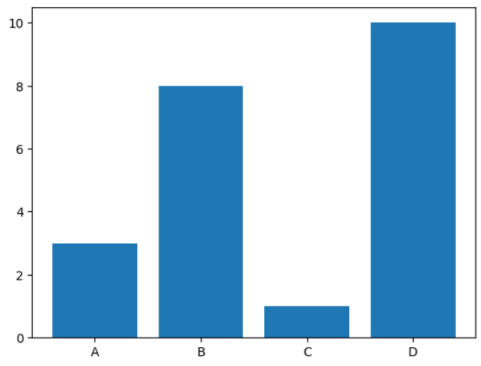
Matplotlib Bars

Creating Bars

With Pyplot, we can use the bar() function to draw bar graphs:

Draw 4 bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```



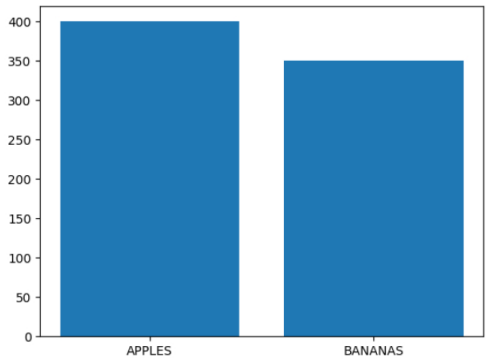
The bar() function takes arguments that describes the layout of the bars.

The categories and their values represented by the first and second argument as arrays.

```
x = ["APPLES", "BANANAS"]
```

```
x = [ APPLES ,  BANANAS ]
y = [400, 350]
plt.bar(x, y)
```

```
<BarContainer object of 2 artists>
```

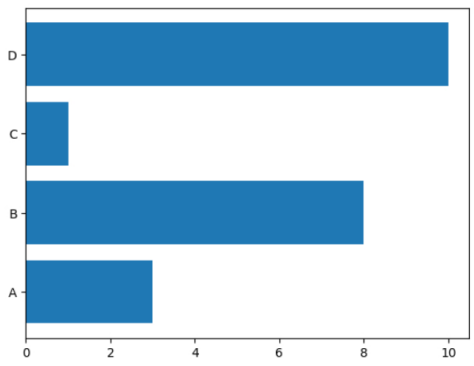


Horizontal Bars

If we want the bars to be displayed horizontally instead of vertically, use the barh() function:

Draw 4 horizontal bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y)
plt.show()
```

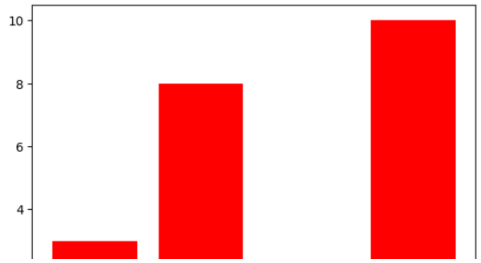


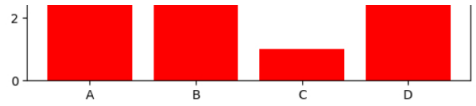
Bar Color

The bar() and barh() take the keyword argument color to set the color of the bars:

Draw 4 red bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "red")
plt.show()
```



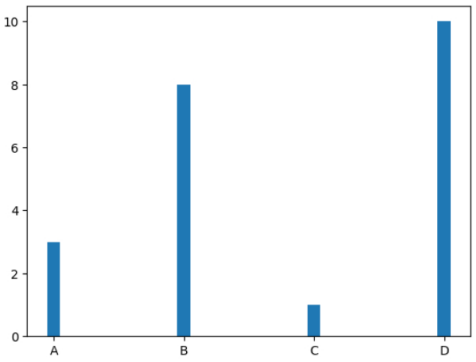


Bar Width

The bar() takes the keyword argument width to set the width of the bars:

Draw 4 very thin bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, width = 0.1)
plt.show()
```



The default width value is 0.8

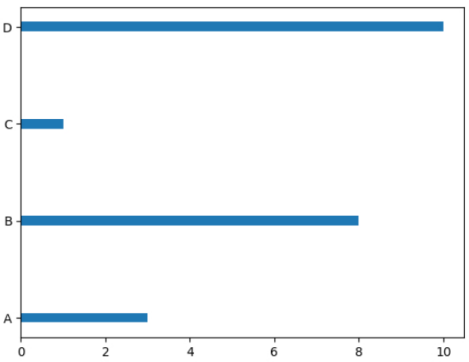
Note: For horizontal bars, use height instead of width.

Bar Height

The barh() takes the keyword argument height to set the height of the bars:

Draw 4 very thin bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y, height = 0.1)
plt.show()
```



Matplotlib Histograms

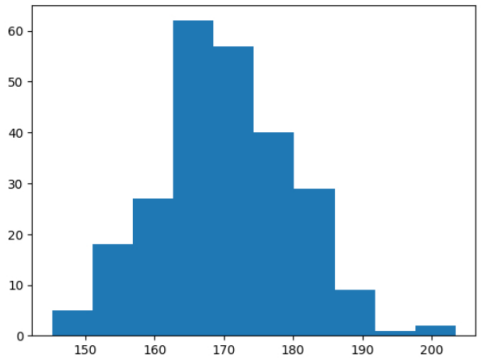
Histogram

A histogram is a graph showing frequency distributions.

It is a graph showing the number of observations within each given interval.

Example: Say you ask for the height of 250 people, you might end up with a histogram like this:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.normal(170, 10, 250)

plt.hist(x)
plt.show()
```



Matplotlib Pie Charts

Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])

plt.pie(y)
```

```
([<matplotlib.patches.Wedge at 0x7dfcccdea150>,
  <matplotlib.patches.Wedge at 0x7dfcccde8a70>,
  <matplotlib.patches.Wedge at 0x7dfcccde88f0>,
  <matplotlib.patches.Wedge at 0x7dfcccdeb530>],
 [Text(0.4993895781431485, 0.9801071621215756, ''),
  Text(-1.0864572197158957, 0.17207762703851495, ''),
  Text(-0.17207745003216413, -1.0864572477508851, ''),
  Text(0.9801074062041268, -0.49938909910391416, '')])
```

```
Start coding or generate with AI.
```

As you can see the pie chart draws one piece (called a wedge) for each value in the array (in this case [35, 25, 25, 15]).

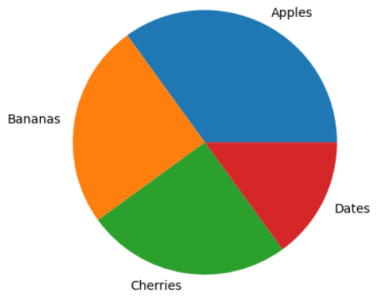
By default the plotting of the first wedge starts from the x-axis and moves counterclockwise:

Labels

Add labels to the pie chart with the labels parameter.

The labels parameter must be an array with one label for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.show()
```



Student Performance Analysis using Histogram

Case Study: Student Performance Analysis

Analyze the performance of students using the given dataset.

Dataset:

Student_ID	Marks	Study_Hours
1	45	2
2	56	3
3	67	4
4	78	5
5	88	6
6	90	7
7	34	1
8	60	4
9	72	5
10	85	6

Q.1 Create a DataFrame

Q.2 Display Basic Information

Display the dataset

Find number of students

Q.3 Analyze Marks Distribution (Histogram)

Plot a histogram of student marks.

Q.4 Study Hours vs Marks

Double-click (or enter) to edit

Solution 1

```
#Create a DataFrame (Students will run this)
import pandas as pd

# Creating student performance dataset
data = {
    "Student_ID": [1,2,3,4,5,6,7,8,9,10],
    "Marks": [45, 56, 67, 78, 88, 90, 34, 60, 72, 85],
    "Study Hours": [2, 3, 4, 5, 6, 7, 1, 4, 5, 6]
```

```
}

df = pd.DataFrame(data)
df
```

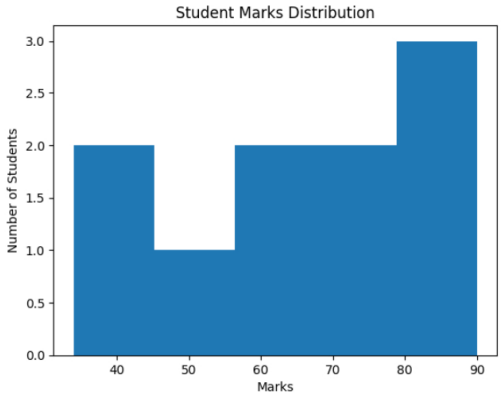
	Student_ID	Marks	Study_Hours
0	1	45	2
1	2	56	3
2	3	67	4
3	4	78	5
4	5	88	6
5	6	90	7
6	7	34	1
7	8	60	4
8	9	72	5
9	10	85	6

Next steps: Generate code with df | New interactive sheet

Solution 2:

[4] ✓ 0s

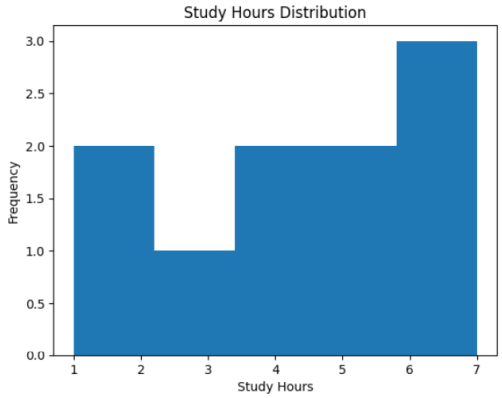
```
##Marks Distribution

import matplotlib.pyplot as plt

plt.hist(df["Marks"], bins=5)
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Student Marks Distribution")
plt.show()
```



Study Hours Analysis

[5] ✓ 0s

```
plt.hist(df["Study_Hours"], bins=5)
plt.xlabel("Study Hours")
plt.ylabel("Frequency")
plt.title("Study Hours Distribution")
plt.show()
```



Students studying more than 5 hours = 4

Most common study hours range: 4–6 hours

Problem 4: Study Hours vs Marks

```
plt.plot(df["Study_Hours"], df["Marks"], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Study Hours vs Marks")
plt.show()
```

Marks increase with increase in study hours

Strong positive relationship observed

Line Chart using Matplotlib

Aim

To plot and analyze a Line Chart using Matplotlib for understanding the relationship between Study Hours and Marks of students.

A line chart is used to display data points connected by straight lines.

It is mainly used to:

Show trends

Analyze relationships

Observe changes over time or order

In data analysis and machine learning, line charts help in identifying patterns between variables.

Dataset Description

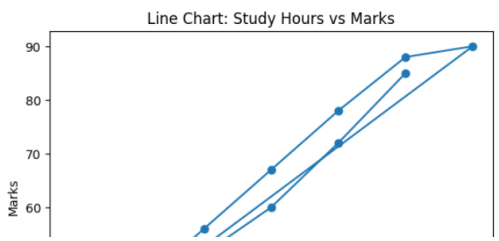
The dataset contains:

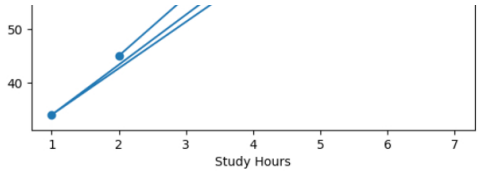
Student ID

Marks obtained

Study hours per day

```
import pandas as pd
import matplotlib.pyplot as plt

# Creating dataset
data = {
    "Student_ID": [1,2,3,4,5,6,7,8,9,10],
    "Marks": [45, 56, 67, 78, 88, 90, 34, 60, 72, 85],
    "Study_Hours": [2, 3, 4, 5, 6, 7, 1, 4, 5, 6]
}

df = pd.DataFrame(data)

# Plotting Line Chart
plt.plot(df["Study_Hours"], df["Marks"], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Line Chart: Study Hours vs Marks")
plt.show()
```





Output

A line chart showing Study Hours on X-axis

Marks on Y-axis

Each point represents a student's performance

Interpretation

As study hours increase, marks also tend to increase

A positive relationship exists between study hours and marks

Students who study more generally score higher marks

Waterborne Disease Trend Analysis

Dataset Description

The dataset shows monthly reported cases of waterborne diseases in a region.

Diseases included:

Cholera

Typhoid

Diarrhea

	Month	Cholera	Typhoid	Diarrhea
0	Jan	5	10	30
1	Feb	8	12	35
2	Mar	12	15	40
3	Apr	20	18	55
4	May	35	25	80
5	Jun	60	40	120
6	Jul	85	55	160
7	Aug	90	60	170
8	Sep	70	45	140
9	Oct	40	30	100
10	Nov	15	20	60
11	Dec	8	15	40

Problem 1: Display the Dataset

Problem 2: Line Chart for Cholera Trend

2.1 In which month are cases highest?

2.1 Why do cases increase during monsoon months?

Problem 3:Compare Multiple Diseases

3.1 Which disease has the highest number of cases?

3.2 During which months do all diseases peak?

3.3 Identify seasonal trends.

Double-click (or enter) to edit

Scatter Chart Analysis

Scatter Chart Analysis of Rainfall vs Waterborne Disease Cases

Aim

To study the relationship between rainfall and diarrhea cases using a scatter chart.

A scatter chart is used to:

Show the relationship between two numerical variables

Identify correlation (positive / negative / no relation)

Detect patterns and outliers

Dataset Description

The dataset contains:

Monthly rainfall (mm)

Number of diarrhea cases

Step 1: Create the DataFrame

[10] ✓ 0s

```
import pandas as pd

data = {
    "Month": ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"],
    "Rainfall_mm": [10, 15, 20, 30, 60, 120, 180, 200, 150, 80, 30, 15],
    "Diarrhea_Cases": [25, 30, 35, 50, 80, 120, 160, 170, 140, 90, 50, 30]
}

df = pd.DataFrame(data)
df
```

	Month	Rainfall_mm	Diarrhea_Cases
0	Jan	10	25
1	Feb	15	30
2	Mar	20	35
3	Apr	30	50
4	May	60	80
5	Jun	120	120
6	Jul	180	160
7	Aug	200	170
8	Sep	150	140
9	Oct	80	90
10	Nov	30	50
11	Dec	15	30

Next steps: Generate code with df | New interactive sheet

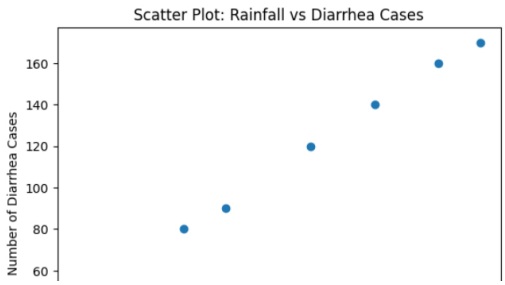
Problem Statement

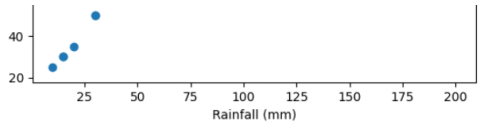
Problem 1: Plot Scatter Chart

Plot a scatter chart between Rainfall and Diarrhea Cases.

[11] ✓ 0s

```
import matplotlib.pyplot as plt

plt.scatter(df["Rainfall_mm"], df["Diarrhea_Cases"])
plt.xlabel("Rainfall (mm)")
plt.ylabel("Number of Diarrhea Cases")
plt.title("Scatter Plot: Rainfall vs Diarrhea Cases")
plt.show()
```





Questions

What type of relationship is observed between rainfall and diarrhea cases?

Do disease cases increase with rainfall?

Identify the month with highest rainfall and cases.

Is scatter chart suitable for trend over time? Why or why not?

ANSWER

A positive correlation is observed

As rainfall increases, diarrhea cases also increase

Highest cases occur during monsoon months (July–August)

Scatter chart is best for relationship analysis, not time trends

A scatter chart is used to show the relationship between two numerical variables.

Problem 1: Temperature vs Cholera Cases

Objective

Study the relationship between water temperature and cholera cases.

[12] ✓ 0s

```python
import pandas as pd

data = {
    "Water_Temperature_C": [18, 20, 22, 25, 28, 30, 32, 33, 31, 28, 24, 20],
    "Cholera_Cases": [5, 8, 12, 18, 30, 45, 65, 70, 55, 35, 15, 8]
}

df = pd.DataFrame(data)
df
```

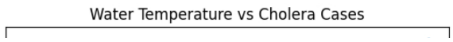
	Water_Temperature_C	Cholera_Cases
0	18	5
1	20	8
2	22	12
3	25	18
4	28	30
5	30	45
6	32	65
7	33	70
8	31	55
9	28	35
10	24	15
11	20	8

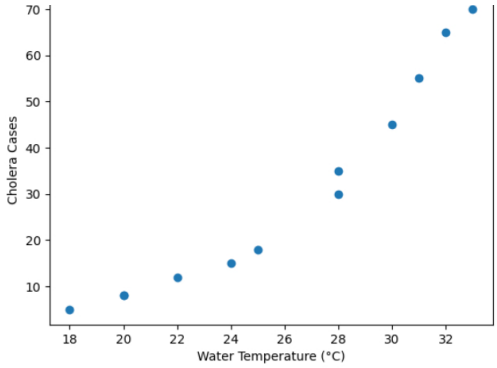
Next steps: Generate code with df | New interactive sheet

Task

[13] ✓ 0s

```python
import matplotlib.pyplot as plt

plt.scatter(df["Water_Temperature_C"], df["Cholera_Cases"])
plt.xlabel("Water Temperature (°C)")
plt.ylabel("Cholera Cases")
plt.title("Water Temperature vs Cholera Cases")
plt.show()
```





[ ] Start coding or generate with AI.

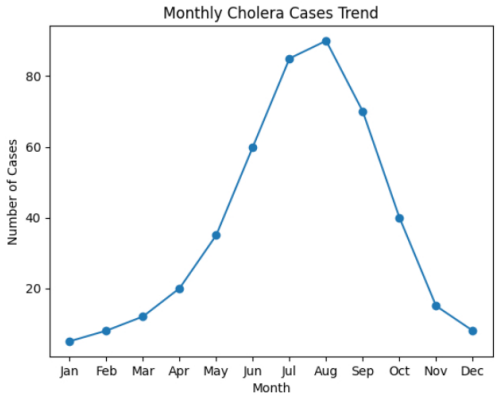
[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[8] ✓ 0s

```
#Problem 2: Line Chart for Cholera Trend
import matplotlib.pyplot as plt

plt.plot(df["Month"], df["Cholera"], marker='o')
plt.xlabel("Month")
plt.ylabel("Number of Cases")
plt.title("Monthly Cholera Cases Trend")
plt.show()
```



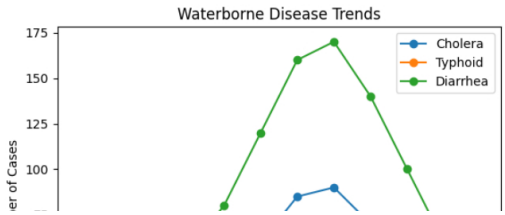
Expected Observation

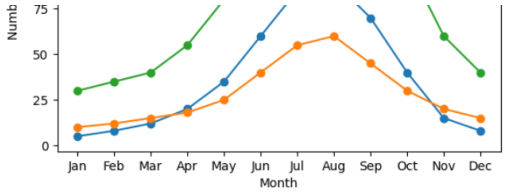
✓ Positive correlation

✓ Higher temperature → more bacterial growth

[9] ✓ 0s

```
#Problem 3: Compare Multiple Diseases (Trend Analysis)
plt.plot(df["Month"], df["Cholera"], marker='o', label="Cholera")
plt.plot(df["Month"], df["Typhoid"], marker='o', label="Typhoid")
plt.plot(df["Month"], df["Diarrhea"], marker='o', label="Diarrhea")

plt.xlabel("Month")
plt.ylabel("Number of Cases")
plt.title("Waterborne Disease Trends")
plt.legend()
plt.show()
```





[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

[ ] Start coding or generate with AI.

Market Trend Analysis – Matplotlib

Problem 1: Monthly Sales Trend (Line Chart)

Objective

Analyze monthly sales trend of a product.

	Month	Sales
0	Jan	120
1	Feb	135
2	Mar	150
3	Apr	165
4	May	180
5	Jun	210
6	Jul	250
7	Aug	270
8	Sep	260
9	Oct	230
10	Nov	200
11	Dec	170

Tasks

Plot a line chart

Identify peak sales month

Comment on seasonal trend

Problem 2: Product-wise Sales Comparison (Bar Chart)

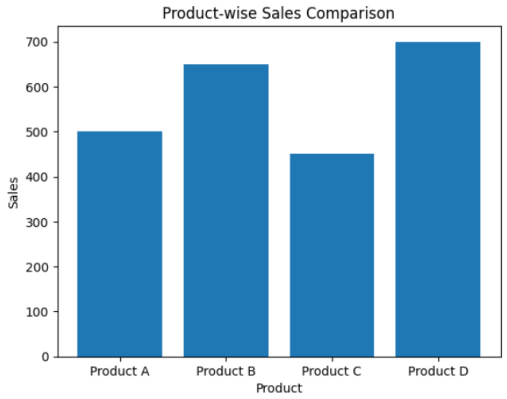
Objective

Compare sales of different products.

	Product	Sales
0	Product A	500
1	Product B	650
2	Product C	450
3	Product D	700

[17]
```
import matplotlib.pyplot as plt
```

```
import matplotlib.pyplot as plt

plt.bar(df["Product"], df["Sales"])
plt.xlabel("Product")
plt.ylabel("Sales")
plt.title("Product-wise Sales Comparison")
plt.show()
```



Market Demand Analysis Using Histogram

Background

A retail company wants to understand the daily demand pattern of a product over a month.

Instead of checking sales day-by-day, the company wants to know:

How demand is distributed

Whether demand is low, medium, or high most of the time

Presence of unusual demand (outliers)

For this purpose, a Histogram is used.

Objective

To analyze the distribution of daily product demand using a histogram in Matplotlib.

Double-click (or enter) to edit

Dataset Description

The dataset represents daily demand (units sold per day) for 30 days.

	Day	Daily_Demand
0	1	95
1	2	100
2	3	105
3	4	110
4	5	98
5	6	102
6	7	108
7	8	115
8	9	120
9	10	125
10	11	130
11	12	128
12	13	122
13	14	118
14	15	112
15	16	108
16	17	105
17	18	100
18	19	98
19	20	96
20	21	94
21	22	97
22	23	103
23	24	107
24	25	113
25	26	118
26	27	122
27	28	126
28	29	130
29	30	135

Double-click (or enter) to edit

Step 1: Create the DataFrame

[18] ✓ 0s

```
import pandas as pd

data = {
    "Day": list(range(1, 31)),
    "Daily_Demand": [
        95, 100, 105, 110, 98, 102, 108, 115, 120, 125,
        130, 128, 122, 118, 112, 108, 105, 100, 98, 96,
        94, 97, 103, 107, 113, 118, 122, 126, 130, 135
    ]
}

df = pd.DataFrame(data)
df
```

	Day	Daily_Demand
0	1	95
1	2	100
2	3	105
3	4	110
4	5	98
5	6	102
6	7	108
7	8	115
8	9	120
9	10	125
10	11	130
11	12	128
12	13	122
13	14	118
14	15	112
15	16	108
16	17	105
17	18	100
18	19	98
19	20	96
20	21	94
21	22	97
22	23	103
23	24	107
24	25	113
25	26	118
26	27	122
27	28	126
28	29	130
29	30	135

Next steps: Generate code with df | New interactive sheet

Problem 1: Understand the Dataset

How many days of demand data are recorded?

What is the minimum and maximum demand?

Line chart: shows trend over time

Scatter plot: shows relationship between variables

Histogram: shows data distribution

Bar chart: compares categories

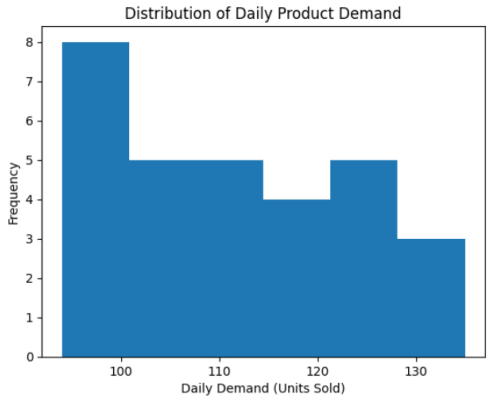
Double-click (or enter) to edit

Problem 2: Plot Histogram of Daily Demand

[19] ✓ 0s

```
import matplotlib.pyplot as plt
```

```
plt.hist(df["Daily_Demand"], bins=6)
plt.xlabel("Daily Demand (Units Sold)")
plt.ylabel("Frequency")
plt.title("Distribution of Daily Product Demand")
plt.show()
```



Interpretation Questions

Which demand range occurs most frequently?

Is the demand evenly distributed?

Are there days with unusually high demand?

Does the histogram show a normal distribution?

Expected Analysis / Observation

Most demand values lie between 100–120 units

Very few days have extremely high demand (130+ units)

Demand distribution is approximately normal

No extreme outliers are present

The histogram clearly shows how daily demand is distributed, helping management understand typical sales levels instead of focusing on individual days.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit